

Interactive Beat Annotation GUI

Table of Contents

Introduction.....	3
Contextualization and Problem Framing.....	3
Research Questions.....	4
State-of-the-Art.....	5
Beat tracking Research.....	5
Beat Annotation Methods.....	7
Existing Tools for Beat Annotation.....	7
HIL Approach for BT.....	8
Audio-to-Score Alignment for BT.....	10
Identified Gaps.....	11
Work Plan.....	12
1 st Iteration of the GUI.....	13
1 st Testing phase.....	15
Intermediate Presentation Preparation.....	15
2 nd Iteration of GUI.....	16
2 nd Phase Testing.....	18
Public Defense Preparation.....	18
Thesis Finalization.....	19
Appendix.....	20
Bibliography.....	20

Table 1: List of abbreviations

GUI	Graphical User Interface
UI	User Interface
UX	User Experience
BT	Beat Track/ing
BA	Beat Annotation/ing
HIL	Human-in-the-Loop
BPM	Beats Per Minute
SV	Sonic Visualizer
SoTA	State-of-the-Art

Introduction

Contextualization and Problem Framing

Music Information Retrieval (MIR) is a research field that develops computational methods to analyze, interpret, and retrieve musical information from audio, examples include: harmony, melody, rhythm, and structural elements. Within MIR, Beat Tracking (BT) focuses on the automatic extraction of musical beat information from recordings. Since the beat provides a fundamental temporal framework for music, BT is a critical component of many MIR systems.

“Many MIR systems first analyze the beats as the starting point, assume the results would be correct, and use them as a unit for further processing.” (Yamamoto, 2021)

The beat is typically defined as a regularly occurring pulse that underlies the temporal organization of a musical piece. However, the perception of the beat can vary across listeners. An objectively definitive beat can only be established if it is explicitly defined by the composer or agreed upon by performing musicians.

BT has been a research focus for decades. Existing algorithms perform reliably for music with steady, predictable tempos, such as Pop music. In contrast, music with expressive and/or fluctuating tempos remains challenging for autonomous beat detection. Examples of music genres that are still difficult to assess are, classical, jazz or world music. These can make the process of beat tracking more complex and ambiguous.

For this reason SoTA BT autonomous algorithms are all machine learning (ML) based. And although there are a few that are self-supervised (Desblancs et al., 2023), these are usually genre specific or are bound to certain musical configurations. For the cases that SoTA BT still can't solve, human-made BA (or revisions) are still the most reliable.

Although there has been a lot of effort in advancing autonomous BT solutions, the same can not be said for human focused BT tools where it seems that they have been stayed largely unchanged for years now.

Research Questions

RQ1. What limitations exist in current beat annotation workflows when applied to challenging music with high expressiveness, such as tempo variability?

RQ2. How can different musical, model and signal representations support and guide the beat annotation process?

A RQ2 é intencionalmente abstrata, em termos práticos poderia ser partida em sub-questões:

- RQ2.1. How can a symbolic score representation, defined in musical time, be aligned with an audio-based representation defined in absolute time?
- RQ2.2. How can an audio spectrogram support accurate manual beat annotation in challenging music?
- RQ2.3. How can deep neural network output activations, representing the probability of beat presence over time, guide the annotation process?
- RQ2.4. Can such activation functions be used to define or support a beat quantization grid during annotation?

RQ3. To what extent does the proposed beat annotation workflow affect annotation time and efficiency when compared with existing tools?

State-of-the-Art

Beat tracking Research

It is clear that BT is one of MIR's most fundamental and most researched fields. As of now BT is considered a "solved" problem for music with steady 4/4 tempo, this includes a majority of western commercially available music (Country, Disco, HipHop, Rock, Metal, Pop etc.). Where SoTA BT algorithms still falter is in music that goes a stray fall from the never changing, 4/4 metric and steady bpm that is the standard for modern western music. In musical genres/examples with fluctuating bpm and complex or unconventional time signatures SoTA BT still falters. This includes music that is "acoustically" performed, meaning, music where the bpm can fluctuate for performative or expressive reasons. Examples include genres such as Classical, Jazz and World Music.

Besides this, the act of assessing the beat can be ambiguous. In some examples, annotators might tap at half, double, or even triple the actual beat. It is also possible for one to feel the subdivision at different levels, and this may impact the beat assessment of the annotator. For these cases, there can only be a "correct" assessment if the recording in question has an underlying musical score that explicitly describes the beat, i.e., classical music.

For these reasons, current SoTA beat tracking methods are predominantly based on neural networks (CNN, TCN's, Transformers). ZeroNS (Desblancs et al., 2023), BeatThis! (Foscarin et al., 2024), and HingeNet (Ru et al., 2025) are just 3 examples of SoTA algorithms that illustrate the complexity of the BT problem:

- ZeroNS was the first self-supervised BT model. It works by analyzing separately the percussive (drums) and melodic parts of a given recording. A problem with this model is that by design is dependent of instrumentation, this model works best for music with drums and clear melodic instrument distinction (voice, guitar, keys, bass etc...). Besides this, ZeroNS performs poorly against other supervised SoTA models.
- BeatThis! Is still considered the SoTA BT model. It works by analyzing spectrogram windows of the recording.
- HingeNet combines different BT models and adds a harmonically aware layer on top. Although HingeNet is newer, it is still F-measure testes show that it is slightly behind BeatThis!.

The performance of BT algorithms is based on an F-measure metric presented in 0.0 to 1.0 ratio or the equivalent in percentage. This measures a BT assessment against a ground-truth assessment of a given dataset. It takes into account correct beats, false positives (extra beats) and false negatives (missed beats), it also allows for a $\pm 70\text{ms}$ window of tolerance for each beat.

BT models are tested against well documented datasets. In Figure 1 I present four that are common to see as benchmark tests for BT:

- GTZAN was specifically designed for musical genre classification. It is a collection of 2000 music excerpts of 30sec each. It covers 10 different music genres, plus 4 and 6 sub-genres of classical and jazz music respectively, each having 100 excerpts. This gives a total of 19h of audio to be tested.
- Ballroom is a dataset consisting a total of 698 total examples. With eight different dance music genres: Cha-Cha, Jive, Quickstep, Rumba, Samba, Tango, Viennese Waltz, and Slow Waltz. BPM ranges from 60-224. As this dance-music, bpm stays mostly the same across the excerpts, for this reason it is expected for SoTA models to perform F-measures > 90%.
- Hainsworth: A dataset consisting of 221 musical examples from a variety of genres, rock/pop, dance, classical, folk and jazz.
- SMC Mirex: Compiled to include pieces that are challenging for SoTA benchmark test. This data set consists of 217 manually-annotated highly precise musical pieces.
-

Figure 1: Tabel comparing the performance (F-measure in %) of SoTA BT models.

Dataset	Zero-Note Samba (2023)	Beat This! (2024)	HingeNet (MERT-based) (2025)
GTZAN	87.5%	89.1%	89.7%
Ballroom	91.1%	97.5%	97.3%
Hainsworth	78.3%	91.9%	91.4%
SMC Mirex	52.0%	62.7%	61.7%

It is important to note the distinction between Beat Tracking (BT) and Beat Annotation (BA).

- BT refers to autonomous beat prediction models.
- BA refers to a human-made annotation, as in using SV for doing a beat assessment. BA might use a BT algorithm as a starting point and then go from there.

Beat Annotation Methods

- Need for correction and shifting operations, ELABORAR MAIS

For music in which autonomous SoTA BT systems aren't efficient enough, relaying on human made BA still might be preferable to the annotator:

“A common method to create beat annotations for music recordings is to let a human annotator tap along with them.” (Driedger et al., 2019)

The problem with this, is that these annotations will probably not be accurate enough even if the user performing them is an experienced musician. There are multiple reasons that stem from perception, human motor noise, jitter and system latency (Driedger et al., 2019). Thus when tapping, there is the need to shift marked beats in order to make them accurate:

“This shifting operation is particularly relevant when correcting tapped beats, which can be subject to human motor noise as well as jitter and latency during acquisition.” (Sá Pinto et al., 2020)

Existing Tools for Beat Annotation

Sonic Visualizer is still the go-to software for scientific and MIR purposed audio analysis. However when it comes to BT, SV is outdated due to its inefficient BT labeling process, that is still very much manual and cumbersome (Yamamoto, 2021). SV also is heavily reliant on VAMP plug-ins, modern SoTA BT algorithms are complex ML applications that may be difficult to implement into VAMP plug-ins and/or there is no direct or valued incentive for making these algorithms available in this way.

Time Instances inside SV (name of the main tool in SV for BT), can be exported and imported as simple .txt files. Thus, for one to benefit from a SoTA BT algorithm, one needs to follow a few steps:

- First, one needs to have access the algorithm in itself which might involve not only browsing/researching, but also probably, accessing some publicly available source-code repository.
- Secondly, one needs to know not only how to compile and run the program, but also export the retrieved data into a .txt file that SV can read.
- Besides this, we also need to take into account that to accomplish these steps, one needs at least, to be minimally literate/experienced with programming.

A problem with this, is that other users, who may not be experienced in programming or literate in MIR research, will rely on the most easily accessible BT algorithm inside SV, and these will probably be Beat Trackers like: BeatRoot (S. Dixon, 2001), Tempo and Beat Tracker - Queen Mary University of London (Stark et al., 2009), and IBT – INESC Beat Tracker (Oliveira et al., 2010) among others. The reason being that, although they don't come pre-installed with SV, they can be added via a native feature that allows to browse and install VAMP plugins without exiting the program (referring to the "Find Transform" SV tool). Crucially it is important to note that, as of the writing of this paper, you will not find any SoTA BT using this SV built in feature, in fact one will only find outdated BT that are at least more then a decade old.

One of the most common autonomous BT assessment mistakes are the Double/Half time errors, in which the algorithm might mark the beat in orders of integer multiplication (or division) from the actually correct beat.

Another common mistake might be inducing multiple closely sequenced markings in the place where only one should be, or the contrary, omitting/skipping beats. For correcting these type of mistakes, current Beat Tracking software only allow correcting beats one-by-one. A simple workaround for this mistake would be to develop a feature in which the user could select a time region of beats and perform an operation such as:

'In this region, that start at xx:xx and ends at yy:yy , the tempo is a steady X bpm with Y/Z timeSignature.' The program would then shift/eliminate/highlight marked beats that deviate from the "user statement".

HIL Approach for BT

In 2021 Yamamoto proposed a Human-In-the-Loop (HIL) approach for BT, the problem is, although this paper seems promising, the software developed for this paper is closed-sourced, property of Yamaha Corp. and thus, it is not likely to be publicly accessible.

The paper also fails to mention how a spectrogram can be beneficial as opposed to the more conventional waveform representation of audio. A waveform can be useful for identifying general differences in energy, on the other hand spectrograms can, besides that, represent information related with spectral energy in time (frequency-energy).

A clear example of this would be: In a waveform, one can distinguish a *ff* section of from a *pp* in a recording. In a spectrogram, one can not only distinguish these, but also understand if a particular section would be chord hit, or an *arpeggio*, along side noticing where these note onsets would be located pitch wise.

Audio-to-Score Alignment for BT

Something that is not very well documented, is the relevance of following along a music score when performing a BA. Besides helping the annotator follow along more accurately the recording, a score might notice the annotator in advance of any bpm or time signature changes. Besides that, when using a Spectrogram-Based view of the recording, one can distinctively align a specific onset to a concrete and exact moment in the recording timeline, this is essential to know if a beat is accurately assessed or not.

Piano Precision (Jiang et al., 2025), is an example of a SV fork that was designed for displaying musical scores alongside audio recordings in a unified graphical interface. This program works surprisingly well for classical Solo Piano pieces. Now there are two question that arrive from this:

- One is that, by following along the music sheet, Beat Tracking autonomously should be as straight forward as making the program align a beat marking with the respective note onsets that are marked in the score. Although Piano Precision already does this with high enough accuracy for Audio-to-Score alignment, for beat tracking adjustments might be needed, yet, it still might give a better starting point than some SoTA beat assesments.
- The other would be to integrate this idea of Audio-to-Score alignment with a Spectral View of the recording. In Piano Precision this is done via a side-window that displays the a single A4 page of the score, one at a time. When the user hits play, a darkened highlight displays the current playback position in the score, while in another window a spectral view of the audio is following along the playback position horizontally. This allows for a musically natural follow-along of the score, mimicking how a musical score is actually read. However, for audio analysis and visualization purposes, it could be beneficial for the score to be aligned and viewed along side the playback position, meaning, horizontally and with note onsets that are aligned with their respective absolute position in the timeline. For BT this could allow the user to “grab a beat” and position it on the it’s corresponding position on top of the spectrogram representation. Also this could be beneficial for more easily detecting beats that should be shifted.

Identified Gaps

Add a synthesis subsection (currently missing)

This is crucial.

Summarize:

- Tooling gap
- Interaction gap
- Accessibility gap
- Visualization gap

This subsection should directly justify your contribution

Work Plan

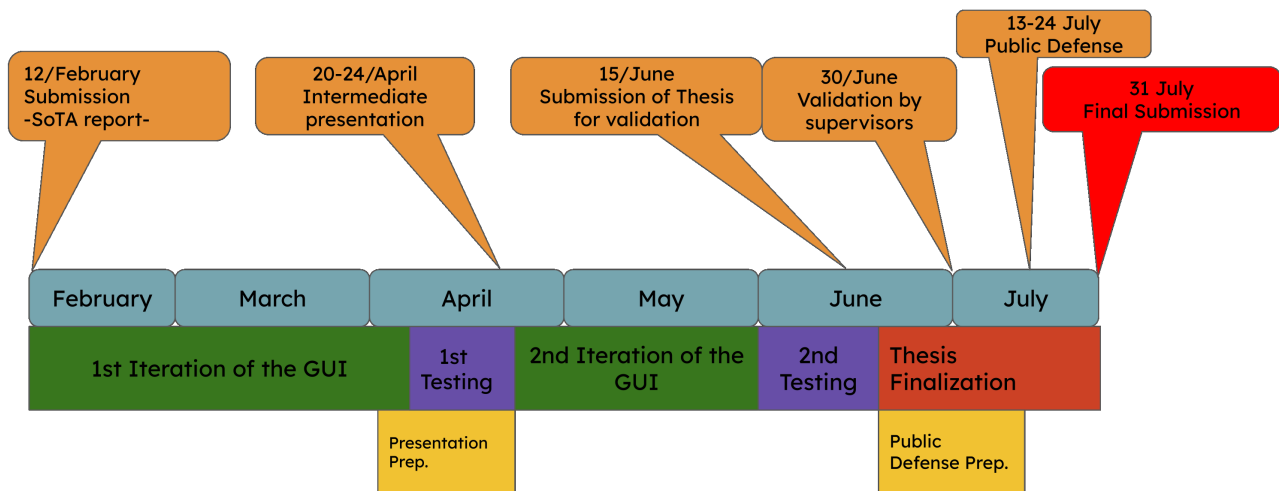


Figure 2: Timeline of planed deadlines for thesis developement.

Table of Contents

Work Plan.....	12
1 st Iteration of the GUI.....	13
1 st Testing phase.....	15
Intermediate Presentation Preparation.....	15
2 nd Iteration of GUI.....	16
2 nd Phase Testing.....	18
Public Defense Preparation.....	18
Thesis Finalization.....	19

Figure 2 shows a timeline with official deadlines and roughly marked self made deadlines. It is important to note that “thesis writing” is not inherently described in the timeline because it is implied that the writing is done along this whole period.

Each of the self imposed deadlines, marked in Figure 2 with the colored squares, have concrete objectives that should be met:

1st Iteration of the GUI

12th February - 5th April

List of main objectives:

- Decision: PianoPrecision fork VS BeatMapJS as starting point
- Basic barebones functional GUI
- Ability to upload a recording with it's .mei score file.
- Basic BA features (add beats via tapping, remove and adjust beats)
- Side-by-side (split horizontally) Spectrogram view of audio and score with roughly aligned score.
- View of note Onsets on top of Spectrogram (with show/hide function)
- Integrated SoTA BeatThis! BT algorithm. On top of the spectrogram (with show/hide function).
- Export beat mapping to .txt file compatible with SV/Audacity.
- Score is set horizontally bellow the spectrogram. No notes need to be displayed. Just the measures with their counter (as described in the designated score): **1 measure should be the length between 2 downbeats.**

The focus of the **1st Iteration** is to have a functional barebones GUI that focuses on the Audio-to-Score alignment functionality. This is not only the most important feature for this work, but it also might be the most challenging to develop. The use of the source code for Piano Precision as a starting point should prove helpful.

During the 1st term of this semester I developed a BA GUI with a Web based interface. Internally named BeatMapJS (link for repo. in Appendix chapter). This GUI was meant as a testing project, but it already has functional:

- Basic features for BA (beat tapping, remove, adjust, add labels and possibility to mark beats as downbeats)
- Working spectrogram view
- BeatThis! BT integration
- Export/Import BA to .txt compatible with SV.
- Other various UI/UX features (region marking and labelling, zoom in/out, spectrogram)

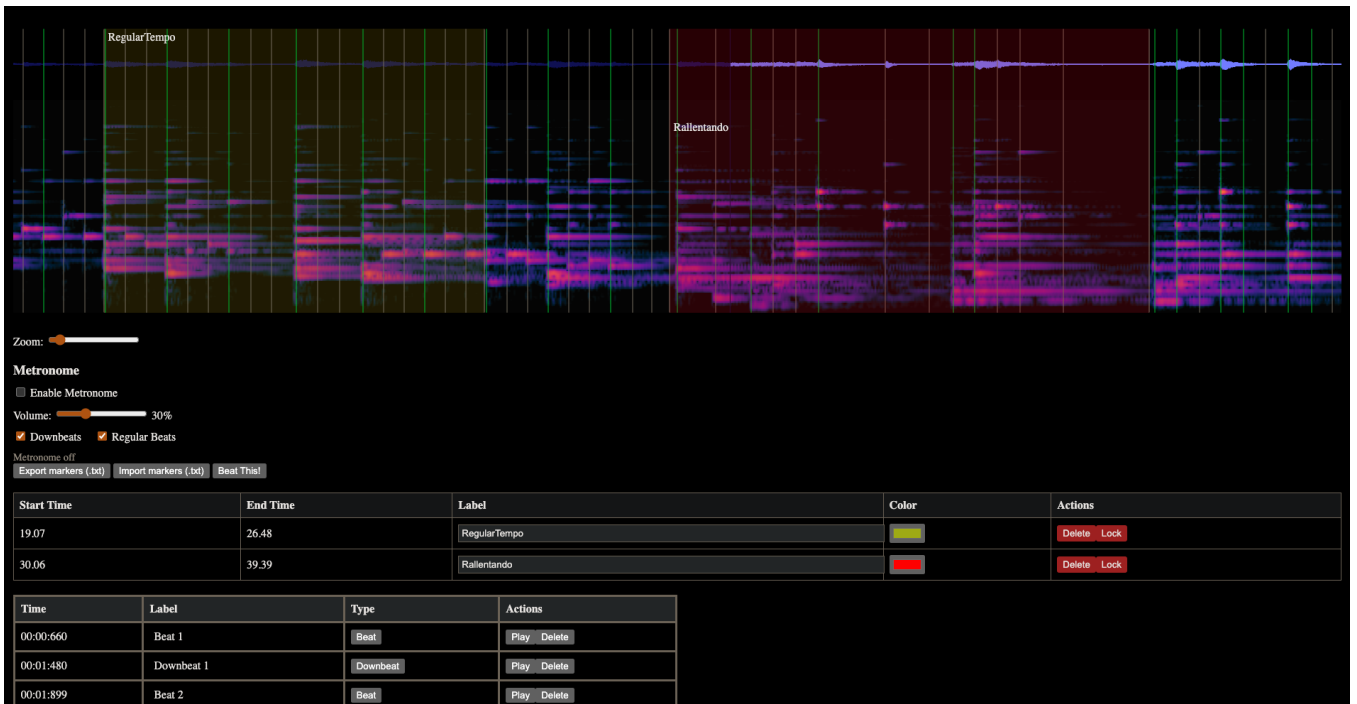


Figure 3: BeatMapJS interface (screenshot)

Firstly it is important to decide if I should use BeatMapJS or Piano Precision (SV fork) as a starting point for the proposed GUI. Using the SV fork, should help with the Audio-to-Score alignment and would probably make into a more robust product due to SV's already pre-made various features. However working with Piano Precision Source code might prove difficult because it is an extensive mainly C++/Qt code base, and honestly... I am not an experienced enough programmer, although not impossible... this makes me fear the worst :')

On the other hand, BeatMapJS is a JavaScript/HTML/python codebase fully developed by me. Although still clunky, it already has essential built-in features. The three main discussion points for reeling on my BeatMapJS repo. are:

1. I would be developing a new web-based GUI (this is not necessarily bad, it is just a statement)
2. Not tacking advantage of SV's and PP already robust codebase and established UI elements.
3. Having to build the score alignment feature from the ground up.

For making the score-to-audio alignment I mean to remove every note and simply represent the score with time signatures and empty measurements (with simple beat markers on top). The reason being , that displaying notes and rhythms on top of the score opens up a to a myriad of interface problems that I describe in more detail in the "[2nd Iteration Phase](#)" chapter. The importance here is to lay the foundation for the audio-to-score alignment for BA. There is a open source tool that might prove helpful in this task, this is the "ScoreWarp" repository (Goebel, 2025). As described in the GitHub repo:

“ScoreWarp is a JavaScript package that warps a score SVG produced by the Verovio engraving engine in order to horizontally align its elements with a given physical performance timeline.”

It is important to note that, during this phase, if every objective is met before the self-imposed deadline, I can and should try to implement 2nd phase development tasks. The more complete the proposed GUI is in this phase, the better.

1st Testing phase

6th of April - 20th of April

List of essential objectives:

- Show developed UI to experienced testers (thesis supervisor, Beat Annotators, teachers, MIR researchers, musicians etc.)
- List possible UI/UX improvements/changes

In this phase I should be able to have an “alpha” version of the proposed GUI. Although being a barebones showcase, it should be in a state where selected users can test it in a controlled way. Meaning, with a selected and pre-made session (i.e. chosen music with already aligned score and BA made). The focus of this testing phase is to serve as proof of concept that Score-to-Audio alignment and spectrogram can be useful tools for BA.

Intermediate Presentation Preparation

1st April - 20th April

List of essential objectives:

- Prepare slide presentation
- Rehearse presentation
- Record/Present Demo of developed GUI

It is essential to prepare a

2nd Iteration of GUI

24th April - 1st June

List of main objectives

- Develop/Implement essential features listed in the 1st Testing Phase.
- Snap to beat functions. Develop user choices: *snap to onset, snap to beat, no snapping*
- Show complete score aligned with the spectrogram (user shall need to adjust portions according to the BA, but that feature should have been already implemented as stated on the 1st phase of development).
- Show problematic beat regions, these should be regions where the beat assessment is irregular or unconventional. User should be able to cycle through these sections quickly in order to flag them as correct or incorrect. The user can then (or later) make the necessary adjustments in problematic flagged regions.
- Develop Region Functions: Delete/Add/Edit multiple beats in a given user defined region. This should serve as a function to among other things, manually correct the half/double/triple BT problem.
- UI/UX various improvements (look and feel, keyboard shortcuts etc.)

If on the 1st phase of development the focus was on introducing a novel BA feature, the Score-to-Audio alignment. On the 2nd phase of development I should be focusing on adding features that actually improve on current BA programs such as SV and Audacity. Among various improvements three stand out:

1. Snap beat feature to align incorrect beats either to onsets or beats assessed by BeatThis! BT model. This should be an on/off function. Can be helpful to enable a more accurate, manually tapped, BA or even to shift BeatThis! beat assessments to close onsets.
2. Add a GUI tool to solve “half/double time” BT assessments.
3. Full Score-to-Audio alignment. On the first stage, I focus on making the score align by making each measure correspond to their real time length. This should be the ground work to then implement the a full score visualization with notes. Although a lot of questions will come from this like:
 - If the score has multiple instruments (ex. orchestra, choral, ensemble etc.) how will it the representation work?

- How will view zoom in/out work with the score allignment? When zooming out, notes and measure can get all scrambled. Do we define the zoom to be limited in a given range for this not to happen?
- If there is a staff or time signature change on the beginning of the measure how should the alignment be made for the measure vs first note of the measure?

The first improvement point (snap beat feature), is one that will benefit from the python library “Shift If You Can” developed by the thesis supervisor (Sá Pinto et al., 2020). This repository is a library for beat shifting related operations. This might be especially helpful for this task, but might also be useful in other aspects of the project (such as the labelling of “problematic” areas).

The second point is one clear improvement on current BA software (SV and Audacity). This is the most common BT error for decades now being commonly referenced in a lot of BT literature: (Dixon, 2001), (Gouyon et al., 2006), (González, 2013), (Foscarin et al., 2024). The solution I envision for this is to let the user define the given region that is affected by this, and simply:

- Set the bpm of that given region and/or set if the assessment is half/double/triplet etc. than it should be. Beats that were off would be eliminated/highlighted so the user can then confirm the alteration. So If a user defines:

“This section is marking double time starting at x beat of x measure and goes back to normal on y beat of y measure.”

If this was the case, then every other beat marking will be eliminated, starting at beat x measure x until beat y measure y.

This would quickly solve the section while at the same time correcting the measurement and beat counting label.

- In addition to this, the user could set if the bpm of that section is regular or if it fluctuates. Meaning beats of that section could either: Be set with a regular interval between them (as set by absolute bpm value). Or if this section has fluctuations in bpm, then the beat would snap to the closest onset or beat prediction.

This section, where the BT seems to be almost correct but with a few false positives (added beats) and/or false negatives (omitted beats) and/or slightly shifted beats every now and again. To correct this, as in the previous example, the user could set if the bpm of this region so that beats either are forced to be equally distanced (as set by user bpm) or snapped into closest onset/beat prediction.

For the third point (full score-to-audio), which might prove one of the most challenging features to implement I still don’t know how to tackle. I believe I first have to accomplish the 1st Iteration and go through the 1st phase of testing to define better what I need to achieve in this topic.

One thing is clear in my mind. The adoption of a segmented BA process UX wise. By this I mean that any change needs to be confirmed by the user in order to be “locked” in the beat assessment. Changes are made in section that are either automatically or manually designated, within these sections, the original/previous states are always shown as a “un-highlighted/darkend” beat markings, until the user submits the given section as “corrected”. The benefit is that the user can always have the previous state as visual comparison against the new markings. This way if the user has a mental mapping of where multiple beats should be than he can more easily assess which tool to use. Also in a given section, the user could cycle through beat snapping, onset snapping, grid snapping etc, while still having the visual cue of the previous state. The user might also find helpful the ability to quickly switch between hearing the old version, and the new possibility. This also turns BA a more systematic process where the user has to complete x amount of tasks thus making BA a more standardized process.

2nd Phase Testing

1st June - 13th June

During this phase thesis doc. should be 85% written by it's end 100% (before revision phase).

List of main objectives:

- Final testing phase of the proposed GUI
- Utilizing testers feedback to make conclusions for thesis
- Listing flaws reported by testers

For this phase to start my GUI needs to be accomplished, I shall not change or add anything in the source code beyond this point. This is the essential phase to deliver a complete thesis report. It is through this phase that I will assume conclusions of the effectiveness of my proposed GUI and it is through it that I will objectively understand my project's strengths and flaw. Besides, this is a essential step to know how a work like this can move forward and contribute to MIR and BT research fields.

Public Defense Preparation

15th June - 24th July

List of essential objectives:

- Prepare slide presentation
- Rehearse Public Defense
- Record/Present Demo of developed GUI

This is it. The final presentation of my work. The place where I can explain in face to face my thesis work, it's relevance and it's impact. It is fundamental that I have a well prepared and well rehearsed Public Defense.

Thesis Finalization

15th June – 31st Jult

List of main objectives:

- Finalize thesis document with supervisor's advised corrections
- Formatting thesis details
- Review whole bibliography
- Prepare project repository for publishing

By the 15th of June (Submission of thesis for validation) a complete thesis should be submitted. Of course there will probably still be points to add or polish but those should come from my supervisor and other academics in the field.

I should take advantage during time, to be sure that my thesis is a polished and complete work.

If we intend for my work to be publicly published, then I shall use this time to make sure that my source-code is well documented and commented, so that future work (either by myself or others) can be facilitated.

Appendix

Link for BeatMapJS repo: <https://github.com/FernandoAze/BeatMapJS.git>
Link for ScoreWarp repo: <https://github.com/iwk-digital/scorewarp>
Link for ShiftIfYouCan repo: <https://github.com/asapsmc/ShiftIfYouCan>

Bibliography

- Desblancs, D., Lostanlen, V., & Hennequin, R. (2023). Zero-Note Samba: Self-Supervised Beat Tracking. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 31, 2922–2934.
- Dixon, S. (2001). *An Empirical Comparison of Tempo Trackers*.
- Dixon, S. (2001). An interactive beat tracking and visualisation system. *Proceedings of the International Computer Music Conference (ICMC)*, 215–218.
- Driedger, J., Schreiber, H., de Haas, W. B., & Müller, M. (2019). Towards Automatically Correcting Tapped Beat Annotations for Music Recordings. *Proceedings of the 20th International Society for Music Information Retrieval Conference (ISMIR)*.
- Foscarin, F., Schlüter, J., & Widmer, G. (2024). Beat this! Accurate beat tracking without DBN postprocessing. *Proceedings of the 25th International Society for Music Information Retrieval Conference (ISMIR 2024)*.
- Goebl, W. (2025). *Let's do the ScoreWarp again! Shifting notes to performance timelines*.
- González, J. R. Z. (2013). *Comparative Evaluation and Combination of Automatic Rhythm Description Systems*.
- Gouyon, F., Klapuri, A., Dixon, S., Alonso, M., Tzanetakis, G., Uhle, C., & Cano, P. (2006). An experimental comparison of audio tempo induction algorithms. *IEEE Transactions on Audio, Speech, and Language Processing*, 14(5), 1832–1844.
<https://doi.org/10.1109/TSA.2005.858509>
- Jiang, Y., Weigl, D. M., & Goebl, W. (2025). Piano Precision: Advancing Music Performance Analysis by Integrating User-Correctable Audio-to-Score Alignment. *Proceedings of the International Computer Music Conference (ICMC)*.

- Oliveira, J. L., Gouyon, F., Martins, L. G., & Reis, L. P. (2010). IBT: A Real-Time Tempo and Beat Tracking System. *Proceedings of the 11th International Society for Music Information Retrieval Conference (ISMIR)*.
- Ru, G., Wang, J., Zhao, J., Wu, Y., Yu, Y., Jiang, N., Wang, W., & Li, W. (2025). *HingeNet: A Harmonic-Aware Fine-Tuning Approach for Beat Tracking* (arXiv:2508.09788). arXiv. <https://doi.org/10.48550/arXiv.2508.09788>
- Sá Pinto, A., Domingues, I., & Davies, M. E. P. (2020). Shift If You Can: Counting and Visualising Correction Operations for Beat Tracking Evaluation. *Extended Abstracts for the Late-Breaking Demo Session of the 21st International Society for Music Information Retrieval Conference (ISMIR)*.
- Stark, A. M., Davies, M. E. P., & Plumbley, M. D. (2009). Real-time beat-synchronous analysis of musical audio. *Proceedings of the 12th International Conference on Digital Audio Effects (DAFx-09)*.
- Yamamoto, K. (2021). Human-in-the-Loop Adaptation for Interactive Musical Beat Tracking. *Proceedings of the 22nd International Society for Music Information Retrieval Conference (ISMIR)*.